Web Music Player

Architecture Documentation

structured after the arc42 template (v8)

Nicolà Widmer · HSLU · Web Programming Lab

2026-09-01

Contents

1 Introduction and Goals

A web-based music player. Users upload MP3 files, organise them into playlists, and play them back on a screen styled as a record player: a vinyl disc that spins while a track plays and a tone-arm that drops on play and lifts on pause. The two self-defined CRUD resources are **Songs** and **Playlists**. The full functional scope and its MoSCoW prioritisation live in `Proposal.typ`; this document describes **how** the system is built and **why**.

1.1 Requirements Overview

- **Songs (CRUD)** — upload `.mp3` (MIME + extension checked, > 20 MB rejected), ID3 tags pre-fill metadata, file on a storage volume + metadata row in PostgreSQL; sortable list view; edit title/artist/album; delete removes row, file, and playlist references.
- **Playlists (CRUD)** — named, ordered collections of songs; a song may sit in several playlists; reorderable; deleting a playlist keeps the songs.
- **Playback** — record-player view, auto-advance, seek, repeat, volume, resume after reload; audio served with HTTP Range support.
- **Authentication** — email + password registration, bcrypt hashes, session cookie; every song and playlist belongs to one user and is invisible to others.

1.2 Quality Goals

Prio	Quality goal	Motivation
1	Turntable-like experience	The playback screen must **feel** like a real turntable — disc spin and tone-arm motion — while still honouring `prefers-reduced-motion`.
2	Smooth audio streaming & seeking	Playback starts without downloading the whole file; seeking works on mobile. Requires HTTP Range (`206 Partial Content`) end to end.
3	Testability & maintainability	Domain logic isolated from Express/PostgreSQL so a use case is unit-testable with no I/O; a clear test pyramid (Jest / Supertest / Playwright).
4	Per-user data privacy	A user only ever sees or changes their own songs and playlists; enforced server-side, not just in the UI.
5	One-command operation	`docker compose up` brings the whole system up on a clean machine with no manual steps and no cloud account.

1.3 Stakeholders

Role	Expectations towards the architecture
Developer / student (Nicolà Widmer)	A structure to build within the time box; guardrails that keep decisions consistent; fast feedback from tests.
Assessor / lecturer (HSLU)	Lab requirements met (≥ 2 self-defined CRUD resources, automated tests, Docker packaging); architecture and decisions documented.
End user / self-hoster	Reliable upload and playback, a private library, a simple setup.
Future maintainer	Understandable module boundaries and ADRs that explain the "why".

2 Architecture Constraints

2.1 Technical Constraints

Constraint	Consequence
Frontend framework: **Angular**	SPA, TypeScript, stand-

	alone components.
Backend: **Express.js**, REST	Node.js/TypeScript HTTP layer; no GraphQL, no SSR.
Database: **PostgreSQL**	Relational metadata store; SQL migrations.
Audio bytes on the **server filesystem** (mounted volume), DB holds the path; S3-compatible storage is an optional (Could) alternative	A storage abstraction is needed so the backend is not hard-wired to `fs`.
Tests: **Jest** (unit), **Supertest** + real PostgreSQL (integration), **Playwright** (E2E)	Ports must be mockable; integration tests need a real DB

	(and MinIO for the S3 path).
Packaging: **Docker Compose**, `docker compose up`	Every runtime dependency is a Compose service; config comes from env vars.
Target: modern evergreen browsers	No IE/legacy shims; `<audio>` + Media Session API assumed available.

2.2 Organisational Constraints

- **Single developer**, fixed academic deadline — scope is protected by the MoSCoW list in the proposal.
- Solo work — architecture must be navigable without a team to ask.

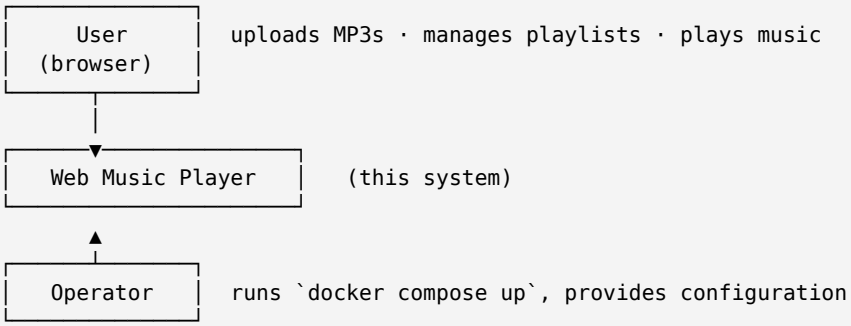
2.3 Conventions

- Backend follows **Domain-Driven Design** (tactical patterns) with a **hexagonal** (ports & adapters) layering; one module per bounded context.
- Significant decisions are recorded as **ADRs** under `Documentation/adr/` (MADR-style, numbered, immutable) and referenced from section 9.
- TypeScript everywhere; ESLint + Prettier; SQL migrations are forward-only.

3 Context and Scope

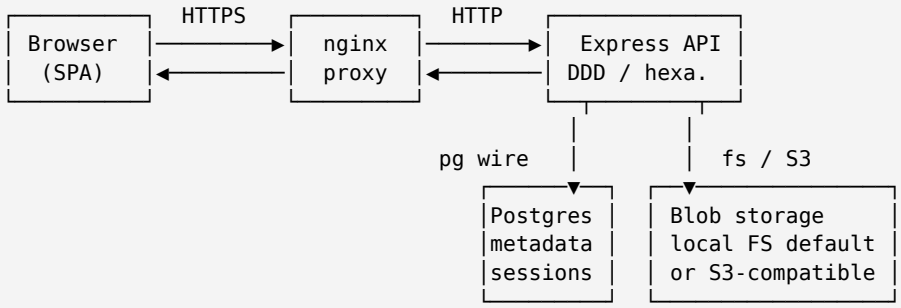
3.1 Business Context

The system is self-contained: the only external actor is the **user** in a browser. There is no third-party integration in the committed scope (no email provider — email verification and password reset are explicitly out of scope). An **operator** deploys and runs the Compose stack.



3.2 Technical Context

Channel	Protocol	Payload / purpose
Browser ↔ nginx	HTTPS	SPA assets (HTML/JS/CSS); REST/JSON API calls; audio stream with `Range` / `206`.
nginx ↔ Express API	HTTP (proxy)	Reverse-proxied `/api/*`; `Range`, `X-Forwarded-*` headers passed through.
Express API ↔ PostgreSQL	TCP (pg wire)	Song & playlist metadata, users, session records.
Express API ↔ Blob storage	filesystem calls *or* S3 HTTP API	Read/write/delete audio objects; range reads for streaming.



3.3 Scope Boundaries

In scope	Out of scope
MP3 upload with validation and ID3 extraction; song & playlist CRUD; record-player playback with seek / repeat / volume / resume; range streaming; session-cookie auth with per-user isolation; optional S3 storage backend.	Email verification & password reset; audio transcoding or non-MP3 formats; sharing / collaboration / social features; native mobile apps; multi-tenant administration; CDN and horizontal scaling (considered only as an evolution path).

4 Solution Strategy

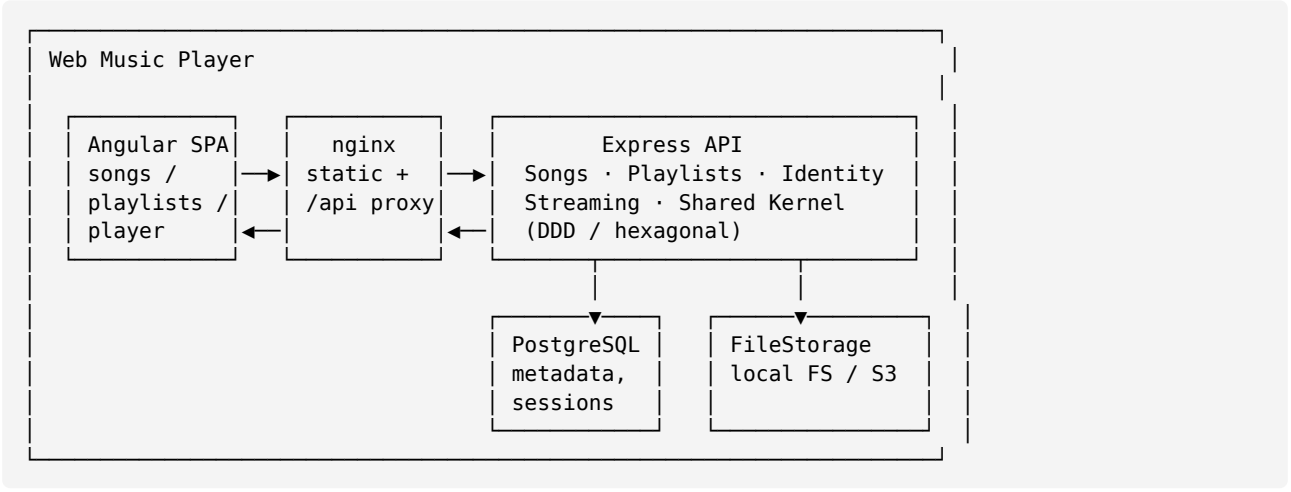
Quality goal	Architectural approach	Ref
Maintainability, testability	**Monorepo monolith**: `backend/`, `frontend/`, Compose at the root — one	`0001-monorepo-monolith`

	deployable, one repo, simple mental model.	
Testability, maintainability	**DDD + hexagonal** backend: one module per bounded context (`songs`, `playlists`, `identity`, `streaming`); the dependency rule points inward; cross-context communication via an in-process domain-event bus.	`0002-ddd-hexagonal-backend`
Streaming performance, one public origin	A **dedicated nginx container** serves the built SPA and reverse-proxies `/api` to Express — static files and `Range` streaming stay off the Node event loop; single origin keeps the session cookie first-party.	`0003-nginx-serves-frontend`
Portability, future multi-node	**PostgreSQL for metadata + pluggable binary storage** behind a `FileStorage` port (local filesystem default, S3-compatible optional).	`0004-metadata-postgres-blob-storage-port`
Per-user privacy	**Server-side session-cookie auth**; bcrypt password hashes; ownership enforced inside use cases (every repository query scoped by	`0005-session-cookie-auth`

	`ownerId`), with an `AUTH_ENABLED=false` mode backed by one implicit local user.	
One-command operation	Everything is a Compose service; 12-factor env config; migrations run on API start; the S3 backend is a Compose **profile**, off by default.	—
Turntable UX	Frontend feature slice `player/` with a single signal-based store as the source of playback truth; CSS animation gated on `prefers-reduced-motion`; `<audio>` element wrapped by an `AudioService`.	—

5 Building Block View

5.1 Level 1 — Whole System



Building block	Responsibility
Angular SPA	All UI and client-side state: song list & forms, playlist management, the record-player view and playback store, auth screens. Talks only to `/api`.
nginx	TLS termination, serving the immutable SPA bundle with cache headers, SPA history fallback, reverse-proxying `/api/*` to the API (including `Range`), enforcing `client_max_body_size 20m`.

Express API	The DDD backend. Exposes the REST API, owns all business rules and persistence. Structured into bounded-context modules over a shared kernel.
PostgreSQL	Durable store for song/playlist metadata, users, and session records.
FileStorage	Binary audio objects. A port with a local-filesystem adapter (default) and an S3-compatible adapter.

5.2 Level 2 — Inside the Express API

Each module is sliced into the same four layers. `domain` depends on nothing; `application` depends on `domain`; `infrastructure` implements `domain` / `application` ports; `interface` (HTTP) depends on `application`. The composition root (`container.ts`) wires concrete adapters chosen from config.

Module	Responsibility & key elements
`shared`	Shared kernel: base classes (`AggregateRoot`, `ValueObject`, `DomainEvent`), the domain-event bus, the `FileStorage` port + adapters, the pg `Pool` and unit-of-work, HTTP error handler and zod validation middleware.
`songs`	Song lifecycle. `Song` aggregate, `AudioFile` / `Id3Tags` value objects, `SongRepository` port, upload/list/update/delete use cases, ID3 reader adapter, `pg` repository, HTTP router. Emits `SongDeleted`.
`playlists`	Ordered collections. `Playlist` aggregate containing `PlaylistItem` entities with a contiguous-ordering invariant, `PlaylistRepository` port, create/rename/add/remove/reorder/delete use cases, a `SongDeleted` handler that drops dangling items.
`identity`	Registration and sign-in. `User` aggregate, `Email` / `PasswordHash` value objects, `UserRepository` port, bcrypt hasher, session middleware, `requireUser` guard.
`streaming`	Serving audio bytes. `StreamSong` use case resolves a storage key and parses the `Range` header; HTTP layer returns `200` or `206` with `Content-Range`. No domain model of its own — an application service over `songs` + `FileStorage`.

5.3 Level 3 — Inside the `songs` Module

Element	Responsibility
`Song` (aggregate root)	Identity, editable metadata (title required, duration read-only), reference to its `AudioFile`. Enforces invariants; produces `SongDeleted` on removal.
`AudioFile` (VO)	`storageKey`, `sizeBytes`, `mimeType`, `durationSeconds` — immutable once uploaded.
`Id3Tags` (VO)	Parsed tag set (title, artist, album, duration, cover); knows the "missing title falls back to filename" rule.
`SongRepository` (port)	`save`, `byId(id, ownerId)`, `list(ownerId, sort)`, `delete`. Always owner-scoped.
`UploadSong` / `ListSongs` / `UpdateSong` / `DeleteSong` (use cases)	Orchestrate validation, ID3 reading, storage and repository calls within one unit of work.
`PgSongRepository` + `SongMapper` (adapters)	Row ↔ aggregate translation over the shared `Pool`.
`MusicMetadataId3Reader` (adapter)	Wraps the `music-metadata` library behind the `Id3Reader` port used by `UploadSong`.

`songs.router` / `songs.controller` / `song.presenter`	Map HTTP ⇄ use-case DTOs; presenter shapes the JSON response.

`playlists` follows the same shape; its distinctive part is that the `Playlist` aggregate owns its `PlaylistItem` list and is the consistency boundary for ordering — reorder and remove operate on the whole aggregate, never on items directly.

6 Runtime View

6.1 Upload a Song (SNG-1 Create)

1. Browser sends `POST /api/songs` as `multipart/form-data` with the `.mp3`.
2. nginx (`client_max_body_size 20m`) forwards to the API.
3. Upload middleware streams the part to a temp file, checks extension + sniffed MIME, rejects > 20 MB.
4. `UploadSong` reads ID3 tags (missing title → filename), opens a unit of work.
5. `FileStorage.put(tempFile) → storageKey` (a random key, never the original filename).
6. `SongRepository.save(song)` writes the metadata row; the transaction commits.
7. On any failure after `put`, the stored object is deleted (no orphans).
8. API responds `201 Created` with the song resource; the SPA adds it to the list.

6.2 Stream and Seek a Track (SNG-2, PB-1)

1. `<audio src="/api/songs/:id/audio">`; the browser issues `GET` with `Range: bytes=START-`.
2. nginx proxies the request, `Range` header intact.
3. `StreamSong` loads the song (owner-scoped), resolves its `storageKey`, parses the range.
4. The API replies `206 Partial Content` with `Content-Range`, `Accept-Ranges: bytes`, `Content-Type: audio/mpeg`, streaming from `FileStorage.getRange(...)` (or a presigned URL redirect for S3).
5. Dragging the seek bar issues a new range request from the new offset.

6.3 Delete a Song That Is in Playlists and Playing (SNG-1 Delete)

1. `DELETE /api/songs/:id` after UI confirmation.
2. `DeleteSong` (unit of work): `SongRepository.delete`, then `FileStorage.delete(storageKey)`.
3. The `Song` aggregate emits `SongDeleted`; after commit the event bus dispatches it.
4. `playlists`' `OnSongDeleted` handler removes matching `PlaylistItem`s from every playlist and closes the ordering gaps.
5. The SPA player store, seeing the current track gone, advances to the next or stops.

6.4 Sign In and an Authorised Request (AUTH-1, AUTH-2)

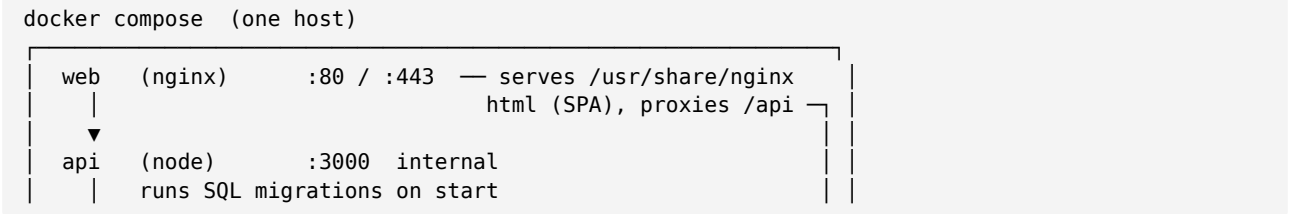
1. `POST /api/auth/session` with email + password.
2. `SignIn` loads the user by `Email`, verifies the bcrypt `PasswordHash`.
3. A session record is written to PostgreSQL; `Set-Cookie` with an opaque session id (`HttpOnly`, `Secure`, `SameSite=Lax`).
4. Later requests carry the cookie; `session` middleware loads it, `requireUser` rejects anonymous calls with `401`.
5. Every use case receives the `ownerId`; repositories filter by it, so another user's resource returns `404`.

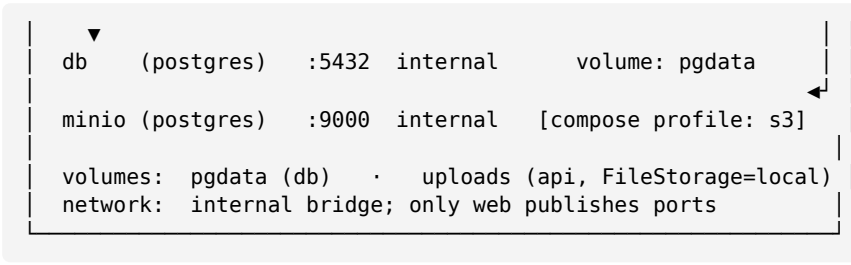
6.5 Resume Playback After Reload (PB-4)

1. While playing, the player store persists `{ playlistId, trackId, positionSec, repeat, volume }` to `localStorage` (debounced).
2. On bootstrap the SPA reads that state, re-fetches the playlist/track metadata, and restores the record-player view **paused** at the saved position.

7 Deployment View

7.1 Single-Node Docker Compose





Node / service	Contains · notes
`web`	nginx + the built Angular bundle. Only service with published ports. Config: upstream `api:3000`, `client_max_body_size 20m`, long `Cache-Control` for hashed assets, `try_files ... /index.html`.
`api`	The Express API image (multi-stage Node build). Reads `DATABASE_URL`, `STORAGE_DRIVER`, `AUTH_ENABLED`, session secret from env. Runs migrations then starts the server. Stateless except for the `uploads` volume when `STORAGE_DRIVER=local`.
`db`	PostgreSQL with the `pgdata` named volume; healthcheck gates `api` start.
`minio`	S3-compatible storage, started only with `--profile s3`; used by the S3 integration tests and by `STORAGE_DRIVER=s3`.

7.2 Notes

- Migrations: a one-shot `api` command on startup keeps `docker compose up` single-step; for production a separate `migrate` service is cleaner.
- Health: `web` depends on `api`, `api` depends on `db` being healthy.
- Secrets: injected via env / Compose `secrets`, never baked into images.

7.3 Evolution to Multi-Node

Switch `STORAGE_DRIVER=s3` (removes the shared-disk requirement), put a CDN in front of `web`, and run several `api` replicas — sessions already live in PostgreSQL, so the API stays effectively stateless. No application code changes, only configuration (see `0004-metadata-postgres-blob-storage-port`).

8 Cross-cutting Concepts

8.1 Domain Model & Tactical DDD

Aggregates (`Song`, `Playlist`, `User`) are the only consistency boundaries. Value objects (`Email`, `AudioFile`, `Id3Tags`, `PasswordHash`) are immutable and self-validating. Repositories are **ports** declared in `domain/`, implemented in `infrastructure/`. Cross-context references are by id only; contexts never import each other's domain code.

8.2 Layering & the Dependency Rule

`interface → application → domain`, with `infrastructure` implementing inward ports. Nothing in `domain/` imports Express, `pg`, `fs`, or `music-metadata`. The single **composition root** `container.ts` is the only place that knows every concrete class; it picks adapters from configuration.

8.3 Cross-context Communication

An in-process synchronous `DomainEventBus`. Aggregates record events; the unit of work publishes them after commit. Current use: `SongDeleted` → playlist cleanup. Keeps `songs` and `playlists` decoupled without a message broker.

8.4 Persistence

One PostgreSQL schema, forward-only SQL migrations. Each use case runs in one transaction (`UnitOfWork`). Mappers translate rows ⇄ aggregates so the domain never sees SQL. No ORM — hand-written queries keep the model explicit and the range/streaming paths predictable.

8.5 Binary Storage & HTTP Range Streaming

`FileStorage` port: `put`, `get`, `getRange`, `delete`, `getUrl?`. Local adapter stores objects under the `uploads` volume by random key; S3 adapter uses the AWS SDK and may return presigned URLs. Streaming always supports `Range`; nginx is configured to forward the header and not buffer audio responses.

8.6 Security & Privacy

- Passwords: bcrypt (cost ≈ 12), only the hash stored.

- Sessions: opaque id in an `HttpOnly` + `Secure` + `SameSite=Lax` cookie; server-side record in PostgreSQL; cleared on sign-out.
- Authorization: enforced **in use cases** (owner-scoped repository calls), not only in middleware; other users' resources return `404`.
- Upload hardening: extension + sniffed MIME, 20 MB cap at nginx **and** API, random storage keys, objects stored outside any web root.
- Input validation: zod schemas at the HTTP boundary; domain re-checks invariants.
- No secrets in the repo or images; env only.

8.7 Error Handling
Domain and application layers return a `Result` type for expected failures (validation, not-found, forbidden). A single Express error handler maps these to `application/problem+json` responses; unexpected errors become `500` and are logged with a request id.

8.8 Configuration & Feature Toggles
12-factor env, validated once at startup with zod (`config/env.ts`). Key switches: `STORAGE_DRIVER=local|s3`, `AUTH_ENABLED=true|false` (false → one implicit local user owns everything), `SESSION_SECRET`, `DATABASE_URL`, `MAX_UPLOAD_MB`.

8.9 UX, Accessibility & Feedback (NFR-1)
Every async action shows loading then a success/error result; every list has a designed empty state. Primary flows are keyboard-operable with visible focus; images carry alt text; contrast meets WCAG AA. The disc-spin animation is disabled under `prefers-reduced-motion`; playback is unaffected.

8.10 Testing Strategy

Level	Scope	Tool
Unit	Domain + application logic, ports replaced by fakes/mocks. No I/O.	Jest
Integration	HTTP → use case → real PostgreSQL (+ MinIO for the S3 path); validation, transactions, ownership, `206` responses.	Supertest
E2E	Browser flows: upload, playlist build, record-player playback, seek, resume.	Playwright

8.11 Build, CI & Code Quality
npm/pnpm workspaces at the root. Pipeline: install → lint → typecheck → unit → integration → E2E → build images. Backend and frontend images are multi-stage; the frontend build output is copied into the nginx image.

8.12 Logging
Structured JSON logger (e.g. `pino`) with a per-request correlation id. Logs upload/delete/auth outcomes and all `5xx`. No audio bytes or credentials logged.

9 Architecture Decisions
This document (arc42) is the **living** architecture description. Individual significant decisions are captured as standalone **ADRs** in `Documentation/adr/`, MADR-style: numbered, immutable once accepted, superseded rather than edited. Each ADR records context, the options considered, the decision, and its consequences. New decisions get the next number and a link back here.

ADR	Decision	Status
`0001-monorepo-monolith`	Single repository, single deployable monolith (`backend/` + `frontend/` + root Compose).	Accepted
`0002-ddd-hexagonal-backend`	DDD tactical patterns with hexagonal layering; one module per bounded context; in-process domain-event bus.	Accepted
`0003-nginx-serves-frontend`	A dedicated nginx container serves the SPA and reverse-proxies `/api`, rather than serving static files from Express.	Accepted

`0004-metadata-postgres-blob-storage-port`	PostgreSQL for metadata; audio bytes behind a `FileStorage` port with local-FS default and optional S3 adapter.	Accepted
`0005-session-cookie-auth`	Server-side session-cookie authentication; ownership enforced inside use cases; `AUTH_ENABLED=false` local-user mode.	Accepted

10 Quality Requirements

10.1 Quality Tree

- **Usability**
  - Turntable metaphor is legible and responsive (Prio 1)
  - Clear loading / success / error feedback; designed empty states (NFR-1)
  - Keyboard operable, WCAG AA contrast, `prefers-reduced-motion` honoured
- **Performance efficiency**
  - Playback starts without full download; seek latency low on mobile (Prio 2)
  - Song list renders quickly with lazy-loaded cover thumbnails
- **Maintainability**
  - Domain logic unit-testable with no I/O (Prio 3)
  - Bounded contexts independently understandable; adapters swappable
- **Security**
  - Strict per-user data isolation (Prio 4)
  - Safe upload handling; hashed passwords; no secrets in the repo
- **Portability / Operability**
  - `docker compose up` on a clean machine, no manual steps (Prio 5)
  - Storage backend switchable to S3 by configuration only
- **Reliability**
  - Delete leaves no orphaned files or dangling playlist entries

10.2 Quality Scenarios

ID	Quality	Scenario (stimulus → response)	Measure
Q1	Performance	User drags the seek bar to a new position on a throttled mobile connection.	Audio resumes from the new position in < 1 s; a single `206` range request, no full-file fetch.
Q2	Security	User A requests `GET /api/songs/{B's id}`.	Always `404`; covered by an integration test for every resource type.
Q3	Operability	`docker compose up` on a machine with only Docker installed.	App reachable on `:80`, migrations applied, no manual step, within 60 s.
Q4	Reliability / Security	Upload of a 25 MB file, or a `.wav` renamed to `.mp3`.	Rejected with `4xx`; no metadata row and no stored object created.
Q5	Maintainability	A new use case is added to the `playlists` module.	It can be fully unit-tested with fake ports; no PostgreSQL needed for that test.
Q6	Usability	OS "reduce motion" setting is on when a track plays.	The disc does not spin; controls and audio behave normally.
Q7	Reliability	A song that sits in 3 playlists and is currently playing is deleted.	Row + file removed, all 3 playlists lose the entry with

			order preserved, the player advances or stops.

11 Risks and Technical Debt

Risk / debt	Impact	Mitigation	Status
No email verification or password reset (out of scope)	Account recovery is impossible; typo'd email locks a user out.	Documented limitation; email is normalised and format-checked at registration.	Accepted
Session store couples auth to PostgreSQL	Extra table + query per request; another thing to migrate.	Acceptable at single-node scale; the store is behind an interface if it needs to move.	Accepted
Local filesystem is the default storage backend	Blocks multi-node until the S3 path is actually exercised.	`FileStorage` port + MinIO integration test kept green in CI.	Mitigated
ID3 parsing on malformed/exotic files	Upload crash or wrong metadata.	Parse in a try/catch; fall back to filename; unit tests with bad fixtures.	Mitigated
No rate limiting on auth endpoints	Online password brute-force.	Add `express-rate-limit` on `/api/auth/*` (Should); bcrypt cost slows guessing.	Open
Upload size capped in two places (nginx + API)	Misconfigured nginx returns a bare `413` the SPA can't explain.	Keep both limits from one env value; friendly client-side precheck.	Open
Single developer, fixed deadline	Scope creep endangers Must stories.	MoSCoW discipline from the proposal; Could items (S3) are genuinely optional.	Accepted

12 Glossary

Term	Definition
Aggregate / Aggregate root	A cluster of domain objects treated as one unit for data changes; the root is its only external entry point and guards its invariants.
Value Object	An immutable, equality-by-value domain type with no identity (e.g. `Email`).
Bounded Context	A boundary within which a domain model and its terms have one precise meaning; here `songs`, `playlists`, `identity`, `streaming`.

Port / Adapter	A port is an interface the domain/application defines; an adapter is a concrete implementation (DB, filesystem, HTTP).
Hexagonal architecture	Ports-and-adapters style: business logic in the centre, all I/O at the edges behind ports.
Composition root	The single place where the object graph is assembled and concrete adapters are chosen (`container.ts`).
Domain event	A record that something significant happened in the domain (e.g. `SongDeleted`), used to decouple contexts.
ADR	Architecture Decision Record — a short, immutable document capturing one decision, its context and consequences.
arc42	A template for architecture documentation, 12 sections; this document.
Range request / 206	HTTP mechanism to fetch part of a resource (`Range: bytes=...` → `206 Partial Content`); enables audio seeking.
Session cookie	An opaque identifier in a cookie that maps to a server-side session record.
ID3	Metadata tags embedded in MP3 files (title, artist, album, cover).
MoSCoW	Prioritisation scheme: Must / Should / Could / Won't.
MinIO	A self-hostable S3-compatible object storage server, used for local dev and tests.
SPA	Single-Page Application — the Angular frontend.